“Try & Score” — Sentence-Scoring API, per category

How the doctor page’s **Try & Score** button scores a single (edited) sentence, and how the call is routed **per clinical category** (cp · le · ed · inc · ius) to the right prompt. Korean mirror: `SENTENCE_SCORING_API_KR.md`. If this diverges from code, the code wins.

What it does

On the doctor dashboard, when a doctor edits a sentence and clicks **Try & Score**, the app sends that sentence + its category to the backend, which scores it 0-5 with GPT-4o (Azure OpenAI) using the **per-domain scoring prompt** and returns a score + explanation. It is an on-the-fly, single-sentence version of the batch AI scoring step.

End-to-end call chain

```
[Doctor page] PhysicianReportsModifiedV41Timothy.tsx
  button → scoreSentence(newSentence, classNumber)        classNumber = TOPIC_TO_CLASS[topicName]
     |
     ▼ hooks/useDoctorData.tsx  (scoreSentence)
  POST /api/backend/doctor/score-sentence   body = { sentence, class_ }
     |  (Next.js proxy → FastAPI)
     ▼ app/Backend/routes_doctor.py  score_sentence()  [POST /api/doctor/score-sentence]
  domain = class_ or "cp"  → domain_map → domain_short (cp/le/ed/inc/ius)
  prompt = load_prompt("scoring", domain_short, "1")   → ai_pipeline/prompts/scoring/<domain>.py
  result = call_llm(client, model, params, prompt, text=sentence)   → Azure OpenAI GPT-4o
     ▼
  { score, sentence, explanation }
```

Request / response

Request (`SentenceScoringRequest`): | field | type | meaning | |—|—|—| | `sentence` | string | the (edited) sentence to score | | `class_` | string (optional) | the clinical category; if omitted the backend defaults to `cp` |

Response (`SentenceScoringResponse`): `{ score: 0–5, sentence, explanation }`.

Per-category routing (the core)

The category name is translated **twice**: the frontend maps the display topic → a long class name (`class_`), the backend maps that → a short domain, which selects the per-domain prompt file.

Display topic (UI)	`class_` sent (frontend `TOPIC_TO_CLASS`)	→ short domain (backend `domain_map`)	scoring prompt file
Cancer Prognosis	`cancer_prognosis`	`cp`	`ai_pipeline/prompts/scoring/cp.py`
Life Expectancy	`life_expectancy`	`le`	`ai_pipeline/prompts/scoring/le.py`
Erectile Dysfunction	`erectile_dysfunction_potency`	`ed`	`ai_pipeline/prompts/scoring/ed.py`
Urinary Incontinence	`continence`	`inc`	`ai_pipeline/prompts/scoring/inc.py`
Irritative Symptoms	`irritative_urinary_symptoms_frequency_urgency_nocturnia`	`ius`	`ai_pipeline/prompts/scoring/ius.py`

The backend `domain_map` is tolerant — it ALSO accepts the short names (`cp` ⋯ `ius`) and numeric class codes (`1` →cp, `2` →inc, `3` →ed, `4` →ius, `5` →le). The active doctor page (V41Timothy) always sends the **long names** above, so the numeric branch is an unused fallback. Unknown/missing `class_` → `cp`.

Scoring parameters (what GPT-4o is called with)

The on-the-fly endpoint hardcodes: - **model** = `settings.azure_openai_model` (Azure deployment) · **api_version** =

`settings.azure_openai_api_version` - **params** = `{ "max_tokens": 4096, "temperature": 0.3, "top_p": 0.4, "seed": 0 }` - **prompt** = `load_prompt("scoring", <domain>, "1")` — prompt **version "1"** (`input_prompt_1`) per domain. - **call** = `call_llm(client, model, params, prompt, text=sentence)` — the same `call_llm` the batch pipeline uses (`ai_pipeline/scoring.py::run_scoring`).

Score rubric (0-5)

`0` no mention · `1` mention, no risk · `2` qualitative only · `3` numeric, no timeline · `4` numeric + timeline · `5` patient-specific estimate + timeline.

Parameter-correctness review (Try & Score) — verified

- `sentence` is sent as `text=` to `call_llm` — correct.
- `class_` (category) is sent and mapped to the correct short domain for all 5 categories; the right per-domain prompt is loaded. No cross-category leakage.
- **Category is type-safe**: `topicName` is typed `TopicName` (a 5-value union) and only ever iterated from `ALL_TOPICS` ; `TOPIC_TO_CLASS` is `Record<TopicName, string>` (covers all five). So `class_` is never `undefined` in the current code — the backend's "default to `cp` " branch is unreachable (defensive only).
- **Consistent with the batch pipeline** : the runtime batch config `ai_pipeline/config.yaml` uses `model_hyper_params: max_tokens 4096, temperature 0.3, top_p 0.4, seed 0` and `prompts.scoring: "1"` for all five domains — **identical** to the on-the-fly endpoint. `model` is `gpt-4o` on both sides.
- ⚠ **One minor divergence** — **`api_version`** : the dashboard `.env` sets `AZURE_OPENAI_API_VERSION=2024-12-01-preview` , while the AI pipeline uses `2024-08-01-preview` (`config.yaml` + `ai_pipeline/.env`). Same model/params/seed, so scores are not expected to differ materially, but aligning the two api_versions is ideal for strict score comparability.

Key files

- Frontend: `app/Webapp/src/components/PhysicianReportsModifiedV41Timothy.tsx` (`TOPIC_TO_CLASS` , call site), `app/Webapp/src/hooks/useDoctorData.tsx` (`scoreSentence` , request/response types).
- Backend: `app/Backend/routes_doctor.py` (`score_sentence` , `domain_map`).
- AI pipeline: `AI_physician_patient_communication/ai_pipeline/scoring.py` (`run_scoring`), `ai_pipeline/utils/prompts.py` (`load_prompt`), `ai_pipeline/prompts/scoring/<domain>.py` (per-domain prompts), `ai_pipeline/llm.py` (`call_llm`).